

APPENDIX TO “RABIT-KV: A PHYSICALLY PACKED, TARGET-BIT-AWARE KV CACHE FOR LLM SERVING”

Anonymous Authors¹

This appendix gives the full tables behind the compact results of the main paper and the details of its accounting, validation, and diagnostics. Section and table references to “the main paper” are to the submitted manuscript.

A FULL CONTEXT-SCALING RESULTS

Table 1 gives every cell of the context-scaling experiment (single request, 32 greedily decoded output tokens, 15 samples per cell, one session, Llama-3.1-8B). Allocator capacity is the same at every prompt length (393,024 tokens for BF16 and 2,074,592 for RABIT-KV).

Prompt	TPOT (ms)				TTFT (ms)	
	BF16	RABIT	Δ	BF16	RABIT	
512	15.33	21.26	+38.7%	32.1	95.6	
2,048	15.56	21.00	+35.0%	59.8	119.9	
4,096	15.60	21.03	+34.8%	144.6	171.3	
8,192	15.72	20.66	+31.4%	409.5	305.2	
16,384	15.43	21.15	+37.1%	1,280.8	687.8	
32,736	15.56	29.38	+88.9%	4,435.0	114,376.8	

Table 1. Context scaling (medians). The 32,736-token prompt is processed as a 16,384-token first chunk and a 16,352-token later chunk.

Two remarks. First, at 8,192 and 16,384 tokens the RABIT-KV TTFT is lower than that of BF16. Both methods compute the first prefill chunk densely, and we have not analyzed the difference; it is not a general prefill advantage, since TTFT is higher at shorter prompts and about 26 times higher at the 32,736-token point. Second, 32,736 tokens is the nominal model limit of our configuration (32,768 including the output tokens), not a demonstrated maximum context. Absolute latencies in this table come from a different session than the matched four-method comparison of the main paper and must not be compared with it.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

B FULL CONCURRENCY RESULTS

Tables 2 and 3 give all points of the concurrency experiment: a closed loop that keeps at most C requests in flight, 256 measured requests per point after an unmeasured conditioning batch, 32 output tokens per request, three trials per point (medians reported), a fresh engine process per point. TTFT here includes closed-loop queueing time and is not comparable with single-request TTFT. No preemption was observed.

C	Throughput (req/s)			TPOT (ms)	
	BF16	RABIT	ratio	BF16	RABIT
1	1.458	1.022	0.70	20	26
4	4.592	1.612	0.35	21	64
8	7.218	1.844	0.26	21	111
16	10.042	2.015	0.20	35	219
32	12.911	2.083	0.16	52	432
64	14.361	1.776	0.12	108	985

Table 2. Concurrency at 2048-token prompts. Both methods sustain every tested concurrency. RABIT-KV enters the later-chunk prefill path for $C \geq 16$.

C	Throughput (req/s)			TPOT (ms)	
	BF16	RABIT	ratio	BF16	RABIT
1	0.937	0.828	0.88	21	28
4	1.801	1.259	0.70	46	82
8	2.101	1.385	0.66	96	162
16	2.282	1.424	0.62	188	312
32	2.346	0.870	0.37	372	1,033
64	2.366 ^a	0.323	—	571	6,487

Table 3. Concurrency at 8192-token prompts. RABIT-KV enters the later-chunk prefill path for $C \geq 4$. ^aBF16 does not reach the offered concurrency: at most 47 requests are in flight in each of the three trials, the allocator bound for this prompt length, whereas RABIT-KV has 64 in flight. The two $C=64$ entries are therefore measured under unequal load, and we give no ratio.

The later-chunk path is entered whenever the product of concurrency and prompt length exceeds the per-step token budget of 16,384 tokens, because the budget is then shared among requests and a prompt is split across scheduler steps.

C PHYSICAL STORAGE ACCOUNTING

Page bytes. For a model with H KV heads of dimension d (a multiple of 32), one 32-token page of one layer stores:

- key payload: $32 \cdot H \cdot d$ elements at 3 bits, $12Hd$ bytes;
- value payload: $32 \cdot H \cdot d$ elements at 2 bits, $8Hd$ bytes;
- four metadata arrays (key minimum, key scale, value minimum, value scale), each with Hd primary values: one per (head, channel) for keys, and one per (token, head, 32-channel group) for values. Each array is stored as Hd 8-bit codes plus one BF16 minimum and one BF16 scale per 64 values, $Hd + 4 \cdot Hd/64$ bytes.

The page therefore occupies $12Hd + 8Hd + 4(Hd + Hd/16) = 24.25Hd$ bytes. Table 4 instantiates this for the two models.

	Llama-3.1-8B	Qwen2.5-7B
Layers, H, d	32, 8, 128	28, 4, 128
Key payload (B)	12,288	6,144
Value payload (B)	8,192	4,096
Metadata, 4 arrays (B)	$4 \times 1,088$	4×544
Page, one layer (B)	24,832	12,416
Per token, one layer (B)	776	388
Per token, all layers (B)	24,832	10,864
BF16 per token, all layers (B)	131,072	57,344

Table 4. Page accounting for the two model geometries.

Allocator-observed capacity. Table 5 lists what the running engine reports. The bytes per token implied by the reported KV budget and block count agree with the layout bytes to within about ten bytes per token in every case. The measured capacity ratio over BF16 is 5.2785 for both models.

Method	Blocks	Tokens	Implied B/token
<i>Llama-3.1-8B (about 47.98 GiB KV budget)</i>			
BF16	12,282	393,024	131,081
FP8	24,549	785,568	65,540
TurboQuant (tested)	37,600	1,203,200	42,818
RABIT-KV	64,831	2,074,592	24,833
<i>Qwen2.5-7B (about 48.21 GiB KV budget)</i>			
BF16	28,208	902,656	57,348
RABIT-KV	148,895	4,764,640	10,864

Table 5. Allocator-observed capacity. “Implied” divides the reported KV budget by the token capacity; it is derived, not measured.

What the capacity does not include. The residual window and the open group of each active sequence, at most 35 tokens per sequence, are kept outside the block pool and

are not part of these numbers. The live memory of a running workload was not measured; only the size of the pool was.

D CORRECTNESS AND REFERENCE CONFORMANCE

Two implementations are tied to one independently written reference implementation of the representation: the serving implementation, through its test suite, and the quality evaluator, through bit-exact conformance checks. Table 6 summarizes the checks.

Check	Result
Serving test suite (layout, page codec, online state, metadata, grouped-query attention, causal chunked prefill), run before the serving experiments	105 tests pass
Evaluator vs. reference, CPU, synthetic states	276 of 276 cases bit-exact, including 56 of 56 token-by-token aging cases
Evaluator vs. reference, GPU, Llama-3.1-8B keys and values (1,024-token prefill)	32 of 32 layers, every field bit-exact
Evaluator vs. reference, GPU, Qwen2.5-7B keys and values	28 of 28 layers, every field bit-exact
Evaluator vs. reference, GPU, Llama-3.1-8B at 16,383 tokens (511 closed pages, 27 open tokens, 4 residual)	32 of 32 layers, 0 differing elements
Online aging, GPU, synthetic geometries	28 of 28 cases per geometry in each run

Table 6. Conformance with the independent reference. All comparisons are made on one device.

CPU and GPU are not bit-identical. The state computed on the CPU and on the GPU from the same keys and values differs slightly. The earliest difference is in the single-precision group scale, which changes a small fraction of codes by one level (for Llama-3.1-8B at 1,024 tokens, 0.074% of key codes and 0.020% of value codes). For this reason conformance is established separately on each device against the reference running on that device, and no cross-device tolerance is defined or used.

Scope. These checks establish that the evaluator computes the intended stored state and that the serving implementation agrees with the reference on its tested cases. They are not a formal verification, and serving outputs were not compared with evaluator outputs end to end.

E QUALITY DETAILS

Continuation perplexity. Each model is scored on 32 consecutive, non-overlapping WikiText-2 test windows of 1,024 context tokens and 128 scored tokens (4,096 scored tokens per arm). Perplexity is the exponential of the total negative log-likelihood divided by the number of scored tokens; the relative difference is computed from the mean per-window difference in negative log-likelihood, with a paired percentile bootstrap over windows. Table 7 gives the distribution over windows.

	Llama-3.1-8B	Qwen2.5-7B
BF16 perplexity	7.515	6.787
RABIT-KV perplexity	7.633	112.39
Relative difference	+1.56%	+1555.9%
Paired 95% CI	[+1.12, +2.05]%	[+1320, +1828]%
Δ NLL, mean	0.0155	2.807
Δ NLL, median	0.0136	2.795
Δ NLL, minimum	-0.0063	1.931
Δ NLL, maximum	0.0590	3.724
Windows worse / better	30 / 2	32 / 0

Table 7. Continuation perplexity by model. Δ NLL is the per-window mean negative log-likelihood of RABIT-KV minus that of BF16.

For Llama-3.1-8B the loss is spread over the windows and not driven by a few: the three windows with the largest difference account for 28% of the summed difference, and the relative difference without them is +1.24%.

Long-context tasks (Llama-3.1-8B). Prompts are truncated to at most 16,384 tokens (the first and last 8,192 tokens of longer inputs). Generation is greedy with at most 16 new tokens for the needle task and 32 for the LongBench tasks.

- *Needle in a haystack.* A secret code is inserted into WikiText-2 filler text at 19 depths (5% to 95%) for contexts of 4,096, 8,192, and 16,384 tokens, 57 cases in total; a case is correct if the exact code is returned. BF16 and RABIT-KV are correct in 19 of 19 cases at each length. The generated text differs between the arms in 23 cases without changing any score. The cases share one needle and one filler construction and are not independent draws, so no confidence interval is given.
- *Passage Retrieval* (200 examples). Both arms score 100.0; no individual score changes (the predicted text differs in 79 examples).
- *HotpotQA* (100 examples of the long-input subset). Table 8 gives both scorers. The primary scorer does not strip English articles before computing token F1;

the secondary scorer is the standard LongBench answer F1. Both were fixed before the run and are computed from the same predictions. No example is excluded.

	Primary	Secondary
BF16 F1	59.30	59.85
RABIT-KV F1	58.27	58.71
Difference (points)	-1.03	-1.14
Paired 95% CI	[-4.24, +2.20]	[-4.30, +2.03]
Identical / worse / better	92 / 5 / 3	93 / 5 / 2
Median paired difference	0.0	0.0

Table 8. HotpotQA under both scorers. The interval spans zero in both cases, which does not establish equivalence at this sample size.

The bootstrap is conditional on the greedy generations of this run and does not model run-to-run variation of GPU arithmetic. Long-context tasks were not run for Qwen2.5-7B.

F PERFORMANCE DIAGNOSTIC

This section describes a profiling diagnostic that we use only to explain the scaling results. It contains no performance result; every latency and throughput number in the paper comes from unprofiled runs.

Setup. Three workloads were profiled on Llama-3.1-8B with the unmodified serving implementation, each with an unprofiled reference run and a BF16 run of the same shape: (1) 2048-token prompts at $C=8$, where the later-chunk pre-fill path is not entered; (2) 8192-token prompts at $C=32$; (3) one 32,736-token request. Request counts were reduced relative to the main experiments. The profiler attached to the engine from outside and timed scheduler steps, attention layers, the execution paths of Section 4 of the main paper, and every kernel launch, in two separate domains (host time and device time spans).

The profiler perturbed execution. Table 9 shows that the profiled RABIT-KV runs were 3.7 to 8.8 times slower than the unprofiled ones. After subtracting the profiler’s own recorded bookkeeping, the remaining time is still 1.35 to 1.63 times the unprofiled time. We therefore report no component percentages: they are not performance measurements. The unprofiled runs, in contrast, agree with the main experiments (1.848 against 1.844 req/s, 0.900 against 0.870 req/s, and a TTFT of 118.3 s against 114.4 s), which confirms that the diagnostic exercised the same workloads.

What the diagnostic indicates. The qualitative picture is consistent across the three workloads, the source code, and one cross-check with a framework profiler.

Workload	Unprofiled (s)	Profiled (s)	Ratio
2048 tokens, $C=8$	51.9	194.2	$3.74\times$
8192 tokens, $C=32$	106.7	670.8	$6.29\times$
32,736 tokens, one request	119.3	1,051.7	$8.82\times$

Table 9. Wall time of the measured phase of the RABIT-KV runs with and without the profiler.

- Work is serialized per request and per token. In decode, each (layer, request, token) triple performs about 10.8 kernel launches: appending and aging, re-quantizing the open key group and re-encoding its metadata, the open-tail attention, the closed-page attention, and a reduction. Inside a later prefill chunk, each (layer, token) pair performs about 6.9 launches. These counts are exact; time shares are not.
- The 32,736-token request executes 523,264 such (layer, token) iterations in its later chunk.
- At 32,736 tokens, the closed-page attention of the later chunk additionally accounts for device work on the order of tens of seconds, so removing the host-side serialization alone would not remove the long-context cost.

We do not claim that these costs are intrinsic to the representation. Removing them requires batching across requests and tokens, which the current implementation does not do.

G STATUS OF EARLIER QUALITY RESULTS

During development, quality was measured with an earlier logical evaluator. A later audit against the reference implementation found that it did not reproduce the representation exactly, in two respects. First, it flattened the value metadata in (head, token, group) order before the second-level quantization, whereas the representation uses (token, head, group) order, so the 64-value metadata groups contained different values. Second, it quantized only the prompt, once, and left the last prompt token and all generated tokens in full precision, whereas in the representation every token ages through the residual window, the open group, and a closed page.

All quality numbers in the main paper come from the corrected evaluator, which is validated as described in Section D. Results obtained with the earlier evaluator are *legacy logical-evaluator results*. They are not pooled with, or compared against, the results of the main paper, and we draw no conclusion from them. In particular, the component studies carried out during development (key and value bit-width, group size, residual length, and metadata precision) exist only under the earlier evaluator and were not

repeated; this is why the main paper does not claim that the individual design choices are optimal. We did not decompose how much each of the two differences contributes to the change in measured quality.

H REPRODUCIBILITY DETAILS

Hardware. One NVIDIA H100 80GB GPU per run.

Serving configuration. A development snapshot of vLLM with our cache type added; eager mode; 32-token blocks; GPU memory fraction 0.82; maximum model length 32,768; per-step token budget 16,384; chunked prefill enabled; prefix caching disabled; BF16 model weights. Latency experiments use 32 greedily decoded output tokens per request (temperature 0, end-of-sequence ignored). Concurrency experiments set the maximum number of sequences to C and start a fresh engine process for every point.

Baselines. BF16: the default cache. FP8: the engine’s per-tensor FP8 (E4M3) cache with default scales. TurboQuant: the upstream vLLM backend, unmodified in our snapshot, with the preset of 3-bit keys and 4-bit values (with norm correction); by the backend’s default the first two and last two layers stay in BF16, leaving 28 quantized layers.

Quality evaluation. Python 3.11, PyTorch 2.11.0 with CUDA 13.0, Transformers 4.48.2, BF16 weights, batch size 1. Models: Llama-3.1-8B-Instruct and Qwen2.5-7B-Instruct at pinned revisions whose file manifests were checked before every run. Data: the WikiText-2 raw test set; LongBench Passage Retrieval (English) and the long-input HotpotQA subset at pinned dataset revisions, with file hashes checked before every run. Confidence intervals: paired percentile bootstrap, 10,000 resamples, fixed seeds.

Protocol. Every quality run was specified before execution (inputs, example selection, scorers, seeds, validity checks, and interpretation rules) and was accepted or excluded by its validity checks; the checks on output quality use only BF16 outputs. One perplexity run was excluded because it failed a validity check; its results are not used. For the quality experiments, model revisions and file manifests, dataset file hashes, protocol settings, and bootstrap seeds are recorded in the experiment artifacts. For the serving experiments, the artifacts record the engine configuration and hashes of the serving source; a model revision is recorded for the Qwen2.5-7B serving runs but not for the Llama-3.1-8B serving runs.